# SUPPORT VECTOR MACHINES

Concise Study Notes for Exam & Interview Preparation

*Maximum-Margin Classifiers | Hard & Soft Margin | Dual Formulation | Kernel Trick | Practice Questions*

# Contents

# Introduction to Support Vector Machines

A **Support Vector Machine (SVM)** is a supervised machine learning algorithm used mainly for **classification** (and also for regression, called SVR). Given a labelled dataset, SVM tries to find the **optimal decision boundary (hyperplane)** that separates classes with the **widest possible margin** — the largest distance between the boundary and the nearest data points of each class.

Unlike algorithms that only care about classifying points correctly, SVM specifically maximizes the gap between classes. This makes it a **maximum-margin classifier**, which tends to generalize well to unseen data.

> **Why SVM matters**
> SVM remains a popular exam topic because it elegantly connects geometry, convex optimization (Lagrangian duality), and the kernel trick — making it one of the few classical ML algorithms with a strong theoretical foundation.

## Key terms you must know:

| Term | Meaning |
|---|---|
| **Hyperplane** | The decision boundary that separates classes. In n dimensions it is an (n−1)-dimensional flat surface. |
| **Margin** | The distance between the hyperplane and the closest data points from either class. |
| **Support Vectors** | The data points lying closest to the hyperplane — they alone determine its position. |
| **Kernel** | A function that implicitly maps data into a higher-dimensional space to make it linearly separable. |

# Geometric Intuition: Margin & Hyperplane

Imagine two classes of points scattered on a 2D plane. Many lines could separate them — SVM picks the **one line that maximizes the distance to the nearest point of each class**. This distance, measured on both sides, is called the margin, and the chosen line is the maximum-margin hyperplane.

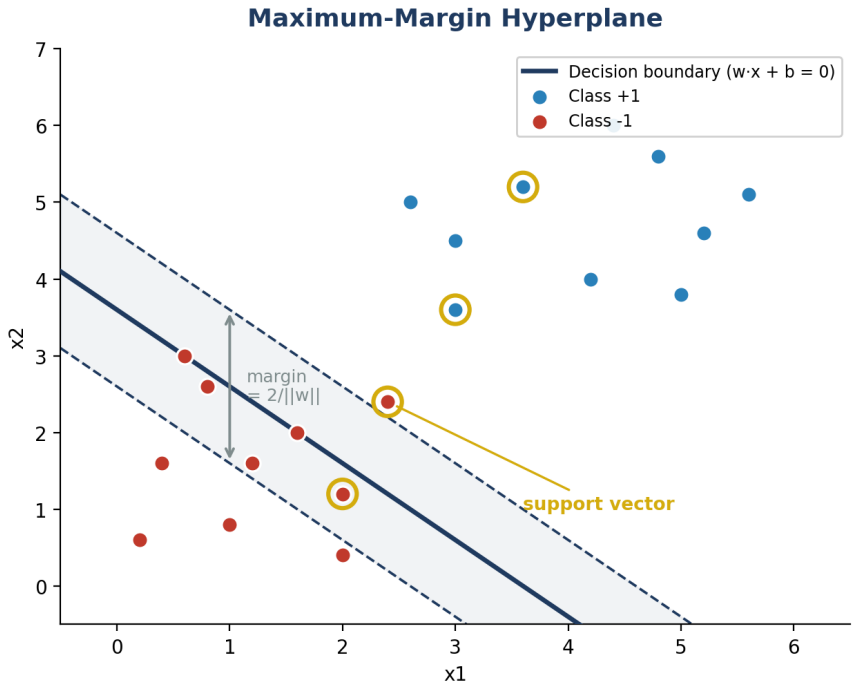


*Figure 2.1 — The solid line is the decision boundary; dashed lines mark the margin. Circled points are the support vectors that pin the margin in place.*

Only the support vectors influence the boundary's position — every other point could be moved (without crossing the margin) and the hyperplane would stay exactly the same. This is what makes SVM relatively memory-efficient at prediction time: it depends only on a subset of training points.

# Mathematical Formulation (Hard-Margin SVM)

## 3.1 Decision Function

The hyperplane is defined by a weight vector **w** and bias **b**. For an input point x, the model predicts:

$$f(x) = \text{sign}(w \cdot x + b)$$

Points are classified as +1 if $w \cdot x + b > 0$, and as –1 if $w \cdot x + b < 0$. The hyperplane itself is the set of points where $w \cdot x + b = 0$.

## 3.2 Margin Width

For a properly scaled w, the two parallel boundary lines touching the nearest points satisfy $w \cdot x + b = +1$ and $w \cdot x + b = -1$. The perpendicular distance between these two lines — the margin — works out to:

$$\text{Margin} = \frac{2}{\|w\|}$$

So maximizing the margin is equivalent to **minimizing ||w||** (or, more conveniently, minimizing $\frac{1}{2}\|w\|^2$ since it is differentiable and convex).

## 3.3 The Optimization Problem (Primal Form)

Assuming the data is linearly separable, every training point $(x_i, y_i)$ with $y_i \in \{+1, -1\}$ must lie on the correct side of the margin. This gives the constrained optimization problem:

$$\underset{w,\, b}{\text{minimize}} \quad \frac{1}{2}\|w\|^2$$

$$\text{subject to} \quad y_i (w \cdot x_i + b) \geq 1 \quad \text{for all training points } i$$

This is a **convex quadratic optimization problem** with linear constraints, which guarantees a unique global minimum — one of the major theoretical strengths of SVM compared to algorithms like neural networks that can get stuck in local minima.

# Soft-Margin SVM & the C Parameter

Real-world data is rarely perfectly separable — there is often noise or overlapping classes. **Hard-margin SVM** would fail here because no hyperplane can satisfy all constraints. **Soft-margin SVM** fixes this by allowing some points to violate the margin, at a cost.

## 4.1 Slack Variables

A slack variable $\xi_i \geq 0$ is introduced per point to measure how far it violates its margin ($\xi_i = 0$ means no violation). The constraint becomes:

$$y_i (w \cdot x_i + b) \geq 1 - \xi_i$$

## 4.2 Modified Objective

$$\text{minimize}_{w,\, b,\, \xi} \quad \tfrac{1}{2} \|w\|^2 + C \cdot \Sigma \xi_i$$

$$\text{subject to} \quad y_i (w \cdot x_i + b) \geq 1 - \xi_i , \quad \xi_i \geq 0$$

Here **$C > 0$** is a regularization hyperparameter that controls the trade-off between a wide margin and classifying training points correctly:

| C value | Effect |
|---|---|
| Large C | Few margin violations allowed → narrower margin, risk of overfitting |
| Small C | More violations tolerated → wider margin, smoother/more general boundary |

**Connection to Hinge Loss**

The term $C \cdot \Sigma \xi_i$ is equivalent to C times the sum of hinge losses, $\max(0, 1 - y_i(w \cdot x_i + b))$. This is why soft-margin SVM is often described as minimizing hinge loss plus an L2 regularization term.

# Lagrangian Dual Formulation

Solving the primal problem directly becomes hard when we want to use kernels. Instead, SVM is usually solved via its **Lagrangian dual**, introducing one multiplier $\alpha_i \geq 0$ per constraint.

## 5.1 Dual Objective

$$\textbf{maximize}_{\alpha} \quad \Sigma\alpha_i - \tfrac{1}{2} \Sigma\Sigma \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$$

$$\text{subject to} \quad 0 \leq \alpha_i \leq C, \quad \Sigma\alpha_i y_i = 0$$

## 5.2 Why the Dual Matters

- The data only appears as a dot product $(x_i \cdot x_j)$ — this is exactly the opening needed for the kernel trick (Section 6).
- At the optimum, most $\alpha_i = 0$. Points with $\alpha_i > 0$ are precisely the support vectors.
- The weight vector can be recovered as $w = \Sigma \alpha_i y_i x_i$ — a weighted sum of only the support vectors.
- This is a convex Quadratic Programming (QP) problem, commonly solved with algorithms such as SMO (Sequential Minimal Optimization).

## 5.3 KKT Complementary Slackness

At the optimal solution, for every point: $\alpha_i \cdot [y_i(w \cdot x_i + b) - 1] = 0$. This means either $\alpha_i = 0$ (the point is not a support vector and lies strictly outside the margin), or the point lies exactly on the margin boundary (a support vector).

# The Kernel Trick

Many real datasets are not linearly separable in their original space. The **kernel trick** solves this by implicitly mapping inputs into a higher-dimensional feature space $\phi(x)$, where a linear separator may exist — without ever computing $\phi(x)$ explicitly.

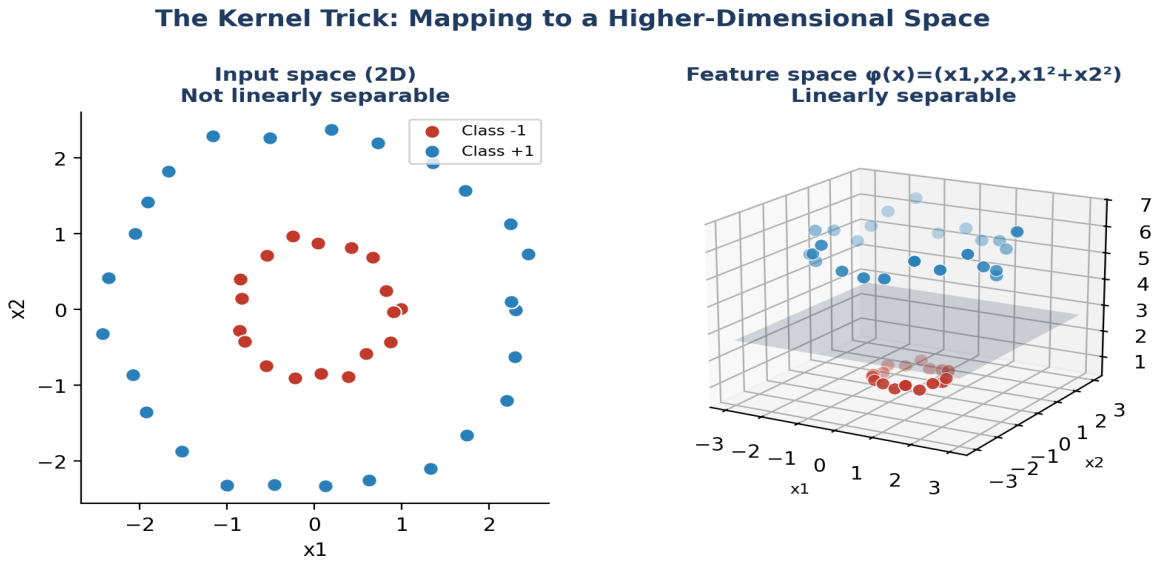


*Figure 6.1 — Concentric classes are inseparable by any straight line in 2D, but become linearly separable once mapped to 3D via $\phi(x) = (x1, x2, x1^2+x2^2)$.*

In the dual objective, the dot product $(x_i \cdot x_j)$ is replaced everywhere by a **kernel function** $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$. Because K can often be computed cheaply without ever forming $\phi(x)$, this avoids the computational cost of working in a very high (even infinite) dimensional space.

$$K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$$

**Mercer's Condition**

Not every function can be a valid kernel. A function K qualifies as a kernel if its corresponding kernel (Gram) matrix is symmetric and positive semi-definite for any set of input points — this is known as Mercer's theorem.

## Common Kernel Functions

| Kernel | Formula | Typical Use Case |
|---|---|---|
| Linear | $K(x_i, x_j) = x_i \cdot x_j$ | Data already linearly separable; text classification with many features |
| Polynomial | $K(x_i, x_j) = (x_i \cdot x_j + c)^d$ | Captures feature interactions up to degree d (e.g. image processing) |
| RBF / Gaussian | $K(x_i, x_j) = \exp(-\gamma \lVert x_i - x_j \rVert^2)$ | Default choice for most problems; handles complex, non-linear boundaries |
| Sigmoid | $K(x_i, x_j) = \tanh(\alpha\, (x_i \cdot x_j) + c)$ | Inspired by neural-network activation; less commonly used |

The RBF kernel is the most widely used default because it can model very flexible, smooth, non-linear boundaries with just two hyperparameters (C and $\gamma$), and it does not require domain knowledge to choose a polynomial degree.

## Key Hyperparameters: C and Gamma

Two hyperparameters dominate SVM tuning, and exam questions love to ask about their trade-offs:

| Parameter | Low value | High value |
|---|---|---|
| C (regularization) | Wider margin, tolerates more misclassification — risk of underfitting | Narrower margin, tries to classify all points correctly — risk of overfitting |
| $\gamma$ (gamma, RBF kernel) | Each point's influence reaches far — smoother, simpler boundary — risk of underfitting | Each point's influence is very local — highly wiggly boundary — risk of overfitting |

**Practical Tip**

C and $\gamma$ are usually tuned together via grid search or random search with cross-validation, since their effects interact — a large $\gamma$ may need a smaller C to avoid severe overfitting, and vice versa.

# Advantages & Disadvantages

| Advantages | Disadvantages |
|---|---|
| Effective in high-dimensional spaces, even when features outnumber samples | Training can be slow on very large datasets (typically $O(n^2)$–$O(n^3)$) |
| Robust to overfitting, especially in high dimensions, due to the margin maximization | Choosing the right kernel and tuning C, $\gamma$ requires careful cross-validation |
| Works well with a clear margin of separation between classes | Does not directly provide probability estimates (needs extra calibration) |
| Memory efficient at prediction time — depends only on support vectors | Less interpretable than simpler models like logistic regression or decision trees |
| Versatile via different kernel functions for linear and non-linear data | Performs poorly with heavy class overlap or very noisy data |

# Real-World Applications

**Text & document classification.** Spam filtering, sentiment analysis, topic categorization — SVM handles high-dimensional sparse text features (e.g. TF-IDF) very well.

**Image classification.** Face detection, handwriting recognition (e.g. digit recognition on MNIST-style data) using pixel or HOG features.

**Bioinformatics.** Protein classification, cancer/gene-expression classification from microarray data where features vastly outnumber samples.

**Face/Object detection.** Historically used (e.g. in early HOG+SVM pipelines) before deep learning became dominant.

**Anomaly/Outlier detection.** One-class SVM variants detect novelties or fraud by learning the boundary of 'normal' data.

# Formula Cheat-Sheet

| Concept | Formula |
|---|---|
| Decision function | $f(x) = \text{sign}(w \cdot x + b)$ |
| Margin width | $2 / \|\|w\|\|$ |
| Hard-margin primal | $\min \tfrac{1}{2}\|\|w\|\|^2$ s.t. $y_i(w \cdot x_i + b) \geq 1$ |
| Soft-margin primal | $\min \tfrac{1}{2}\|\|w\|\|^2 + C \cdot \Sigma \xi_i$ s.t. $y_i(w \cdot x_i + b) \geq 1 - \xi_i$ |
| Dual objective | $\max \Sigma \alpha_i - \tfrac{1}{2} \Sigma\Sigma \alpha_i \alpha_j y_i y_j K(x_i, x_j)$ |
| Recovering w | $w = \Sigma\, \alpha_i y_i x_i$ |
| RBF kernel | $K(x_i, x_j) = \exp(-\gamma \|\|x_i - x_j\|\|^2)$ |
| Polynomial kernel | $K(x_i, x_j) = (x_i \cdot x_j + c)^d$ |
| Hinge loss | $\max(0, 1 - y_i(w \cdot x_i + b))$ |

# Practice Questions

Short-answer and conceptual questions — try answering before checking the hints.

**Q1. Why does SVM maximize the margin instead of just finding any separating hyperplane?**

*Hint: think about generalization to unseen/test data and robustness to small perturbations.*

**Q2. What are support vectors, and why are they the only points that matter for the final model?**

*Hint: recall the KKT condition $\alpha_i[y_i(w \cdot x_i + b) - 1] = 0$ and how $w$ is reconstructed.*

**Q3. Derive why the margin equals $2/\|w\|$.**

*Hint: compute the distance between the two parallel planes $w \cdot x + b = 1$ and $w \cdot x + b = -1$.*

**Q4. Explain the role of the slack variable $\xi_i$ in soft-margin SVM.**

*Hint: $\xi_i$ measures how far a point is allowed to violate the margin or be misclassified.*

**Q5. What happens to the decision boundary as $C \to \infty$? As $C \to 0$?**

*Hint: $C \to \infty$ approaches hard-margin behaviour; $C \to 0$ allows almost unlimited violations.*

**Q6. Why is the dual formulation important for using kernels?**

*Hint: in the dual, data only appears as a dot product $x_i \cdot x_j$, which can be replaced by $K(x_i, x_j)$.*

**Q7. What is the effect of increasing $\gamma$ in an RBF kernel?**

*Hint: $\gamma$ controls how far the influence of a single point reaches — link it to overfitting/underfitting.*

**Q8. Is SVM still a useful algorithm to learn given the rise of deep learning? Justify briefly.**

*Hint: think about small/medium structured datasets, interpretability, and theoretical guarantees.*

**Q9. Name two valid kernel functions and state Mercer's condition for a function to be a valid kernel.**

*Hint: the kernel (Gram) matrix must be symmetric and positive semi-definite.*

**Q10. How does multi-class classification typically work with SVM, which is inherently binary?**

*Hint: research the 'one-vs-rest' and 'one-vs-one' strategies.*

> **Study Tip**
> Before an exam, make sure you can: (1) sketch the margin diagram from memory, (2) write the primal and dual objectives, (3) explain the kernel trick in one sentence, and (4) state the effect of C and $\gamma$. These four cover the large majority of typical SVM exam questions.